Curso
Tendencias Aplicadas de Ingenería de Software

Frameworks para aplicar Asincronismo.
Mosquitto (MQTT) / ActiveMQ (JMS) / Kafka.

Middleware orientado a mensajes

- Un Message Oriented Middleware (MOM) es una infraestructura de software que permite intercambiar mensajes entre aplicaciones de manera desacoplada.
- Su función principal consiste en actuar como intermediario entre productores y consumidores de información.
- Los componentes fundamentales son:
- **Productor**
  - Aplicación que genera mensajes o eventos.
- **Broker**
  - Componente encargado de recibir, almacenar y distribuir mensajes.

Middleware orientado a mensajes

- **Consumidor**
  - Aplicación que recibe y procesa mensajes.
- **Canal**
  - Mecanismo lógico utilizado para transportar mensajes.
- El uso de un broker permite que productores y consumidores evolucionen independientemente, facilitando la integración de sistemas heterogéneos.

Criterios para evaluar tecnologías de mensajería

- La selección de una plataforma de mensajería no debe realizarse únicamente por popularidad o tendencias tecnológicas.
- Entre los criterios más relevantes se encuentran:
- Rendimiento
  - Cantidad de mensajes procesados por unidad de tiempo.
- Latencia
  - Tiempo requerido para entregar un mensaje.
- Persistencia
  - Capacidad para almacenar mensajes de forma durable.

Criterios para evaluar tecnologías de mensajería

- Escalabilidad
  - Facilidad para aumentar capacidad mediante la incorporación de recursos.
- Disponibilidad
  - Capacidad del sistema para continuar funcionando ante fallos.
- Complejidad operativa
  - Esfuerzo requerido para instalación, monitoreo y mantenimiento.

Introducción a MQTT

- MQTT (Message Queuing Telemetry Transport) es un protocolo de mensajería extremadamente ligero diseñado para comunicaciones entre dispositivos con recursos limitados.
- Fue creado originalmente para entornos donde la conectividad es costosa, inestable o presenta restricciones de ancho de banda.
- Sus principales características son:
  - Bajo consumo de recursos.
  - Tamaño reducido de encabezados.
  - Comunicación Publish/Subscribe.
  - Soporte para conexiones persistentes.
- Actualmente es uno de los protocolos más utilizados en soluciones IoT e Industria

Mosquitto como broker MQTT

- Mosquitto es uno de los brokers MQTT más utilizados en entornos académicos e industriales.
- Su función consiste en recibir mensajes publicados por productores y distribuirlos a los suscriptores correspondientes.
- Entre sus características destacan:
  - Implementación Open Source.
  - Bajo consumo de memoria.
  - Autenticación de usuarios.
  - Persistencia configurable.
- Su simplicidad operativa lo convierte en una alternativa muy utilizada para proyectos de telemetría y monitoreo.

Arquitectura de comunicación en MQTT

- La arquitectura MQTT está basada completamente en el patrón Publish/Subscribe.
- Los dispositivos emisores publican información en tópicos específicos y los consumidores se suscriben a aquellos tópicos de interés.

Sensor -> Broker MQTT -> Dashboard

- Los productores y consumidores no necesitan conocerse mutuamente.
- Esta característica reduce significativamente el acoplamiento entre componentes y facilita la incorporación de nuevos consumidores sin modificar aplicaciones existentes.

Caso de uso de MQTT en IoT

- Consideremos una organización que opera miles de sensores distribuidos geográficamente para monitorear variables ambientales.
- Cada sensor transmite periódicamente:
  - Temperatura.
  - Humedad.
  - Presión atmosférica.
  - Calidad del aire.
- En este escenario MQTT resulta adecuado debido a:
  - Bajo consumo de ancho de banda.
  - Mínimo uso de batería.
  - Simplicidad de implementación.
- Por estas razones MQTT se ha convertido en un estándar de facto para soluciones IoT.

Limitaciones de MQTT

- A pesar de sus ventajas, MQTT no fue diseñado para resolver todos los problemas de mensajería empresarial.
- Entre sus principales limitaciones se encuentran:
  - Capacidades limitadas de almacenamiento histórico.
  - Escalabilidad restringida para grandes volúmenes históricos.
- Por esta razón suele complementarse con otras plataformas especializadas cuando los requisitos de negocio aumentan.

Introducción a JMS

- Java Message Service (JMS) es una especificación estándar del ecosistema Java que define una interfaz común para sistemas de mensajería.
- Es importante destacar que JMS no es un producto ni un broker.
- JMS define:
  - APIs.
  - Modelos de comunicación.
  - Contratos de interoperabilidad.
- Las implementaciones concretas son proporcionadas por plataformas como ActiveMQ, IBM MQ o ActiveMQ Artemis.

ActiveMQ como implementación de JMS

- Apache ActiveMQ es una de las implementaciones más conocidas del estándar JMS.
- Está orientada principalmente a entornos empresariales donde la confiabilidad y la integridad de los mensajes son requisitos fundamentales.
- Sus capacidades incluyen:
  - Colas y tópicos.
  - Persistencia.
  - Transacciones.
  - Balanceo de carga.
  - Clustering.
- Es ampliamente utilizado en sistemas bancarios, gubernamentales y corporativos.

Arquitectura de ActiveMQ

- ActiveMQ actúa como intermediario entre sistemas empresariales que necesitan intercambiar información de manera confiable.
- Cada sistema puede evolucionar independientemente mientras el broker garantiza la entrega de los mensajes.
- Esto reduce considerablemente el impacto de cambios tecnológicos y facilita la integración de aplicaciones legadas.

Caso de uso empresarial de ActiveMQ

- Supongamos una institución financiera que procesa miles de operaciones diariamente.
- Cada transacción debe:
  - Registrarse.
  - Validarse.
  - Auditarse.
  - Almacenarse.
- La pérdida de un único mensaje podría generar problemas legales o financieros.
- En este contexto ActiveMQ proporciona las garantías necesarias para asegurar la integridad de la información intercambiada.

Ejemplo de ActiveMQ/Producer

1. Se crea una conexión hacia el broker ActiveMQ.
2. Se inicia la conexión para permitir intercambio de mensajes.
3. Se crea una sesión JMS.
4. Se obtiene una referencia a la cola llamada ordenes.
5. Se crea un productor asociado a dicha cola.
6. Se construye un mensaje de texto.
7. El mensaje es enviado al broker.
8. ActiveMQ almacena el mensaje hasta que algún consumidor lo procese.

En este ejemplo se muestra una aplicación que actúa como productor de mensajes. Su responsabilidad es conectarse al broker ActiveMQ y enviar una nueva orden a una cola denominada ordenes

Ejemplo de ActiveMQ/Producer

```
import javax.jms.*;
import org.apache.activemq.ActiveMQConnectionFactory;

public class Productor {

    public static void main(String[] args) throws Exception {

        ConnectionFactory factory =new ActiveMQConnectionFactory("tcp://localhost:61616");

        Connection connection = factory.createConnection();

        connection.start();

        Session session =connection.createSession(false,Session.AUTO_ACKNOWLEDGE);

        Queue queue = session.createQueue("ordenes");

        MessageProducer producer = session.createProducer(queue);

        TextMessage message = session.createTextMessage("Nueva orden creada");

        producer.send(message);

        System.out.println("Mensaje enviado");

        session.close();
        connection.close();
    }
}
```

Ejemplo de ActiveMQ/Consumer

- Cuando el consumidor ejecuta:
  - consumer.receive();
- queda esperando hasta que exista un mensaje disponible.
- Si la cola contiene:
  - Orden #1
  - Orden #2
  - Orden #3
- el consumidor recibirá la primera orden disponible.
- Una vez procesada y confirmada mediante ACK, ActiveMQ elimina el mensaje de la cola.
- Resultado
  - Antes:
    Orden #1
    Orden #2
    Orden #3
  - Después:
    Orden #2
    Orden #3

Ejemplo de ActiveMQ/Consumer

```
import javax.jms.*;
import org.apache.activemq.ActiveMQConnectionFactory;

public class Consumidor {

    public static void main(String[] args) throws Exception {

        ConnectionFactory factory = new ActiveMQConnectionFactory("tcp://localhost:61616");

        Connection connection =  factory.createConnection();

        connection.start();

        Session session = connection.createSession(false,Session.AUTO_ACKNOWLEDGE);

        Queue queue =session.createQueue("ordenes");

        MessageConsumer consumer = session.createConsumer(queue);

        while(true){

            TextMessage message = (TextMessage) consumer.receive();

            System.out.println("Mensaje recibido: "+ message.getText());
        }
    }
}
```

Introducción a Apache Kafka

- Apache Kafka es una plataforma distribuida de procesamiento de eventos diseñada para manejar enormes volúmenes de datos en tiempo real.
- A diferencia de los brokers tradicionales, Kafka se basa en el concepto de almacenamiento persistente de eventos.
- Su diseño prioriza:
  - Escalabilidad.
  - Rendimiento.
  - Durabilidad.
  - Procesamiento distribuido.
- Actualmente es una de las tecnologías más utilizadas en arquitecturas modernas basadas en eventos.

El concepto de log distribuido

- Kafka almacena los eventos en estructuras denominadas logs distribuidos.
- Cada evento recibe un identificador secuencial llamado offset.
  - Offset 0
  - Offset 1
  - Offset 2
  - Offset 3
  - Offset 4
- Los eventos permanecen almacenados durante períodos configurables independientemente de que hayan sido consumidos.
- Esto permite reprocesar información histórica cuando sea necesario.

Componentes fundamentales de Kafka

- La arquitectura de Kafka se compone de varios elementos especializados.
- **Producer**
  - Genera eventos.
- **Broker**
  - Almacena y distribuye eventos.
- **Topic**
  - Representa una categoría lógica de mensajes.
- **Partition**
  - Permite dividir la carga de trabajo.
- **Consumer Group**
  - Facilita el procesamiento paralelo y escalable.
- La combinación de estos componentes permite alcanzar niveles de rendimiento muy superiores a los brokers tradicionales.

Caso de uso de Kafka en comercio electrónico

- Una plataforma global de comercio electrónico puede generar millones de eventos diarios.
- Entre ellos:
  - Compras.
  - Pagos.
  - Cancelaciones.
  - Envíos.
  - Consultas de productos.
- Kafka permite centralizar estos eventos y distribuirlos simultáneamente hacia múltiples sistemas consumidores, como motores analíticos, sistemas antifraude y plataformas de monitoreo.

Selección tecnológica según escenario

- Utilizar **Mosquitto** cuando:
  - Existan sensores o dispositivos IoT.
  - La conectividad sea limitada.
  - Se requiera bajo consumo energético.
- Utilizar **ActiveMQ** cuando:
  - Se integren aplicaciones empresariales.
  - Existan requerimientos transaccionales.
  - La confiabilidad sea prioritaria.
- Utilizar **Kafka** cuando:
  - Existan grandes volúmenes de eventos.
  - Se requiera analítica en tiempo real.
  - Se adopten arquitecturas orientadas a eventos.

Ejemplo de Kafka/Producer

- Kafka utiliza un enfoque diferente basado en eventos persistentes.

- El siguiente productor publica una nueva orden en un tópico denominado ordenes.
- El productor:
  - Se conecta al cluster Kafka.
  - Selecciona el tópico ordenes.
  - Construye un evento.
  - Envía el evento al broker.
  - Kafka almacena el evento en un log distribuido.

- A diferencia de ActiveMQ, el evento no desaparece cuando es consumido.

Ejemplo de Kafka/Producer

```
import org.apache.kafka.clients.producer.*;
import java.util.Properties;

public class KafkaProducerExample {

    public static void main(String[] args) {

        Properties props = new Properties();

        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,"localhost:9092");

        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringSerializer");

        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringSerializer");

        KafkaProducer<String,String> producer = new KafkaProducer<>(props);

        ProducerRecord<String,String> record =
            new ProducerRecord<>(
                "ordenes",
                "ORD-001",
                "Nueva orden creada");

        producer.send(record);

        producer.close();
    }
}
```

Ejemplo de Kafka/Consumer

- ¿Qué hace el consumidor?
  - Se registra en un Consumer Group.
  - Se suscribe al tópico ordenes.
  - Consulta periódicamente nuevos eventos.
  - Procesa los eventos recibidos.

- Concepto importante
  - El consumidor no elimina eventos.
  - Simplemente avanza su posición de lectura.

Ejemplo de Kafka/Consumer

```
import org.apache.kafka.clients.consumer.*;
import java.time.Duration;
import java.util.*;

public class KafkaConsumerExample {

    public static void main(String[] args) {

        Properties props = new Properties();

        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");

        props.put(ConsumerConfig.GROUP_ID_CONFIG,"grupo-facturacion");

        props.put( ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
             "org.apache.kafka.common.serialization.StringDeserializer");

        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");

        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG,"earliest");

        KafkaConsumer<String,String> consumer = new KafkaConsumer<>(props);

        consumer.subscribe(Collections.singletonList("ordenes"));

        while(true){

            ConsumerRecords<String,String> records = consumer.poll(Duration.ofMillis(100));

            for( ConsumerRecord<String,String> record: records){

                System.out.println(record.value());
            }
        }
    }
}
```

Ejemplo de Kafka/Consumer

- Kafka almacena cada evento con un identificador incremental denominado offset.
  - Offset 0 -> Orden #100
  - Offset 1 -> Orden #101
  - Offset 2 -> Orden #102
  - Offset 3 -> Orden #103
  - Offset 4 -> Orden #104
- Supongamos que el consumidor procesó hasta:
  - Offset 3
- Kafka registra:
  - Última posición leída = 3
- Sin embargo los eventos siguen almacenados.
- Esto permite:
  - Reprocesamiento.
  - Recuperación ante errores.
  - Auditoría.
  - Analítica histórica.

Caso práctico comparativo

- Supongamos una orden de compra:

```
{
  "id": 1001,
  "cliente": "Juan Perez",
  "monto": 250
}
```

- Con ActiveMQ: la orden se envía a una cola:
  - cola-pagos

- Un único consumidor procesa el pago.
- Cuando finaliza: mensaje eliminado

Caso práctico comparativo

- Con Kafka: la orden se publica en:
  - topic ordenes
- Consumidores independientes:
  - Facturación.
  - Inventario.
  - Dashboard.
- Todos reciben el mismo evento.
- Además, pueden releer eventos históricos cuando sea necesario.